

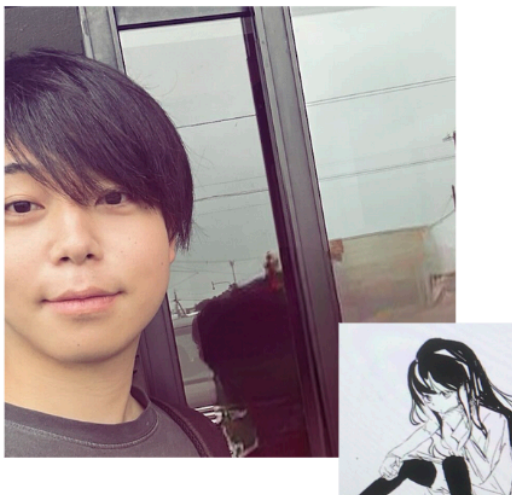


全てのコンテキストを集約し**真の仕様駆動**を実現

プロジェクト全体の開発を高速化する方法

クラスメソッド株式会社 リテールアプリ共創部 マッハチーム

Ruon - 高垣龍平



- 部署
 - クラスメソッド株式会社
 - リテールアプリ共創部
 - 24新卒
- 名前
 - 高垣龍平
 - X:るおん (@ruonp24)

経歴

- 学生時代：アパレル系自社開発企業（インターン）
 - toC向けWebアプリ
- 2024年4月～：クラスメソッド株式会社
 - Webアプリ新規開発・運用保守
 - LINEミニアプリ開発

技術スタック

- フロントエンド（React）
- バックエンド（AWS Lambda）
- インフラ（AWS CDK）
- LINE

実績

- LINE API Expert
- 2025 Japan AWS Jr. Champions



1. AIと仕様駆動の現状
2. コンテキストをローカルに集約する
3. 開発スタイルとドキュメント整備
4. さらなる高速化の秘訣
5. まとめ

AIと仕様駆動の現状

AIにコードを書かせる前に「仕様」を先に作る開発手法

代表的なツール・手法

- **Kiro** - AWSが開発したspec駆動のAI IDE
- **Spec-kit** - specファイルを使った仕様駆動開発ツール
- **Claude Code** - CLAUDE.mdやドキュメントを読み込む
- **Cursor** - .cursorrules / rules / notepads

共通するコンセプト

`spec/` `design/` `plan/` などのディレクトリに仕様を記述し、AIに読み込ませる

実際のプロジェクトはエンジニア1人とAIだけで完結しない

多くのステークホルダーが存在する

- お客様（発注者）
- PM / ディレクター
- デザイナー
- 他社ベンダー（API連携先など）
- 社内の他チームメンバー



本当に必要なコンテキストとは？

従来のSpec駆動が扱う範囲

- 設計書・シーケンス図
- API仕様書
- DB設計
- Figmaデザイン
- 既存コードベース
- 要件定義書

これらも全てコンテキスト

- Backlogの課題・決定事項
- MTGでの合意内容
- 先方のスケジュール
- 使用できる予算
- 自社のリソース状況
- 過去の議事録・見積もり

spec/ だけに仕様を書いても、プロジェクト全体のコンテキストがAIに伝わらない

結果として...

- AIが背景を理解せず的外れな提案をする
- 「なぜこの仕様なのか」がコードに反映されない
- 手戻りが発生し、結局時間がかかる

「真の仕様駆動」とは何か

Vibeコーディング

思いつきで実装
動けばOK
仕様は後付け



Spec駆動（従来）

計画を構造化
spec/design/planを作成
しかし機能仕様のみ



真の仕様駆動

全てのコンテキストを参照
決定経緯・制約・背景も把握
実案件に耐える品質

全ての会話・やり取り・決定履歴を参照可能にし、
本番環境で顧客の要望を満たす実装を実現すること



Kiro等の仕様駆動ツールは「計画の構造化」に過ぎず、
実案件の複雑さには対応できない



コンテキストエンジニアリングのエッセンスを加えることで、
初めて「実案件に耐える仕様駆動」が実現できる

コンテキストをローカルに集約する

情報はいろいろな場所に散らばっている

- **Backlog** - 課題・Wiki
- **GitHub** - Issue・PR・コード
- **Teams/Slack** - MTG・チャット
- **Google Drive** - 共有資料
- **Figma** - デザイン
- **Notion** - ドキュメント

AIエージェントの現状

- 各ツールへのAPI接続が必要
- MCPの設定が複雑
- レスポンスが遅い
- コンテキスト制限に引っかかる
- リアルタイム取得のオーバーヘッド

プロジェクト全体のリソースを **ローカルファイル** として集約
→ AIエージェントが直接ファイルを読み込む
→ 高速かつ確実にコンテキストを取得

オレオレ簡易RAG

ローカル環境でAIおよび人間が**全文検索**できる
情報の一元化が実現

```
apparel-membership-card-context/  
├── MTG/                                # 議事録 (Teamsの自動文字起こしを配置)  
│   ├── 20251105/  
│   │   ├── 20251105_minutes.md        # 議事録  
│   │   └── 20251105_キックオフ.docx   # 文字起こし元  
├── 共有資料/                          # 要件整理・見積もりなど (Google Driveからインポート)  
├── Backlog/                           # Backlogからエクスポートされたデータ  
│   ├── documents/                     # 仕様書・シーケンス図など  
│   │   ├── DB設計.md  
│   │   ├── API設計.md  
│   │   ├── 機能要件.md  
│   │   └── IF仕様書.md  
│   └── issues/                        # Backlog課題  
├── GitHub/                           # GitHubからエクスポートされたデータ  
│   ├── issues/                       # Issue定義ファイル  
│   ├── src/                          # ソースコード 🔗 サブモジュール  
│   └── wiki/                         # GitHub Wiki 🔗 サブモジュール  
└── Figma/                            # Figmaからエクスポートされたデザインデータ
```

💡 `src/` と `wiki/` は別リポジトリをサブモジュールで参照

→ コンテキストリポジトリと開発リポジトリを分離しつつ、ローカルで一元管理

開発

- Issue駆動の実装
- コードレビュー
- テスト作成
- リファクタリング

開発以外

- 要件定義の整理
- 見積もり作成
- 資料作成
- 議事録生成
- 進捗レポート作成
- IF仕様書の作成
- シーケンス図 (Mermaid)
- Backlogコメント返信

1. 全文検索が可能

- AIも人間も同じファイルを検索
- grepやripgrepで高速検索

2. ローカル環境で編集可能

- AIエージェントが直接ファイル編集
- バージョン管理も容易

3. API/MCPが不要

- コンテキスト取得が高速
- レート制限を気にしない
- オフラインでも作業可能

4. プレビューが容易

- Markdownはプレビュー表示
- CSVは拡張機能でExcel風に確認

デメリット

リアルタイムにリモートの状態が更新されない

→ 手動で同期する必要がある

対策：package.jsonにコマンドを用意

```
{
  "scripts": {
    "issue:update": "./sync-github-issues.sh --repo owner/repo",
    "backlog:sync": "backlog-exporter all --domain $DOMAIN ...",
    "rules:generate": "rulesync generate"
  }
}
```

ツール	同期方法	備考
GitHub	<code>gh</code> コマンド	Issue/PR/Wikiの取得・更新
Backlog	<code>backlog-exporter</code>	課題・Wikiのエクスポート
Google Drive	手動コピー	重要資料のみ
Teams	文字起こしをコピー	MTG後に配置
Figma	MCP経由	ローカル同期困難

Figmaは例外

ローカルに落とすのが難しいため、**開発時にMCP経由でリモート状態を読む**

将来的には全てリモート状態になるだろう

- MCPの成熟
- AIエージェントの性能向上
- リアルタイム同期の標準化

しかし、現状のLLM/AIエージェント性能では...

ローカル集約が最も現実的で効率的

一部手動更新があるが、ある程度割り切ることが重要

開発スタイルとドキュメント整備

rulesyncとは

複数のルールファイルから統合されたルールファイルを生成するツール

メリット

- `.cursor/rules/` `.claude/` など複数のAIツールに対応
- カスタムコマンドの共有
- チーム全体で統一されたルール

```
pnpm dlx rulesync@latest generate
```

コマンド	用途
<code>create-minute.md</code>	Teams文字起こしから議事録を自動生成
<code>generate-issue-summary.md</code>	PBIの進捗サマリーを生成
<code>structured-commits.md</code>	作業内容から構造化されたコミットを作成
<code>create-pull-request.md</code>	コミットログからPRを自動作成
<code>update-issue-from-request.md</code>	実装後にIssueを詳細化
<code>create-branch.md</code>	Issue番号からブランチを作成

例：議事録生成

```
/create-minute 20251105_キックオフ
```

→ Teams文字起こしから構造化された議事録Markdownを生成

Issue進捗サマリー生成

`/generate-issue-summary`

PBI（Product Backlog Item）の進捗状況をサマリーとして自動生成

- 完了数、進行中、未着手を一覧化
- 定例MTGの報告資料に活用

Backlog課題の完了サマリー

`/generate-backlog-close-summary`

- 課題をクローズする際の完了報告文を自動生成
- 何を実装したか、どう確認したかを整理
 - コピペでBacklogに貼り付け

ポイント：AIが生成 → 人間がコピペで貼り付け

Backlog・Notion・Slackなど、AIが直接アクセスできないツールへの反映も簡単

基本フロー

1. Backlog/ドキュメントからGitHub Issueを生成
 - └ AIがBacklog課題や設計書を読み込んでIssue作成
2. Issueからブランチを切る
 - └ `/create-branch #123`
3. 実装を進める
 - └ AIがIssue内容を参照しながらコード生成
4. 構造化コミット → PR作成 → Issue更新
 - └ `/structured-commits` → `/create-pull-request` → `/update-issue-from-request`

考えてみてください

- 仕様書を別途作成・維持しようとする、追従が難しく陳腐化しやすい
- ドキュメントは書いた瞬間から陳腐化が始まる
- コードが変更されてもドキュメントが更新されない
- いつの間にか「嘘のドキュメント」になってしまう

これは多くの現場で共通する課題では？

より筋の良いアプローチ

従来	提案
仕様書を別途作成	GitHub Issueに仕様を記載
設計書を別途管理	PRに設計意図を記載
変更履歴を別管理	コミットメッセージが履歴
ナレッジを別ツール	GitHub Wikiに集約

コードと一緒にバージョン管理され、レビューのコンテキストも含まれる

CursorのカスタムスラッシュコマンドでGitHub IssueやPull Requestを品質の高いドキュメントとして自動作成する

<https://dev.classmethod.jp/articles/cursor-slash-commands-auto-generate-issue-pr/>

コミット → PR → Issue の流れでドキュメント品質を保つ

1. /structured-commits
 - └ 作業内容を回想し、関連変更ごとにグループ化してコミット
 - └ コミットメッセージ群がPRの変更履歴として機能
2. /create-pull-request
 - └ コミットログからPR本文を自動生成
 - └ 「やったこと」に具体的な実装内容を記述
3. /update-issue-from-request
 - └ 実装後にIssueを実装内容に合わせて更新・詳細化
 - └ PRへのリンクを追加

さらなる高速化の秘訣

テンプレートリポジトリを事前に用意する

含めておくべきもの

- 認証認可周り - LINE Login, Cognito等
- アーキテクチャ - Clean Architecture, Feature-Based等
- インフラ構成 - CDK, Terraform等
- エラーハンドリング - 共通エラーレスポンス
- ドメインモデル - ビジネスロジックの設計パターン
- CRUDシステム - TODOアプリ的な基本APIエンドポイント

各案件ごとにcloneしてカスタマイズ

品質の一定化

- 毎回同じ品質でスタート
- ベストプラクティスが組み込み済み
- セキュリティ対策が最初から

開発速度向上

- 既存コードの再利用
- 一貫性のあるコード生成
- AIが既存コードを参照可能

作業削減

- 環境構築の省略
- 毎度作るutil関数が不要
- 設定ファイルのコピー不要

```
project-root/  
├── web/           # フロントエンド (React + Vite)  
├── server/        # バックエンド (Lambda + Express)  
├── infra/         # インフラ (AWS CDK)  
├── shared/        # 共通関数・型定義  
└── e2e/          # E2Eテスト (Playwright)
```

React

Lambda

AWS CDK

TypeScript

pnpm workspace

なぜモノレポ？

- **型の共有**: フロント・バック・インフラ間で型定義を共有
- **一貫性**: 同じリンタールール、同じテストフレームワーク
- **AIの理解**: 全体構成を把握しやすい

web/ - Feature-Based

```
src/  
├── features/  
│   ├── facility-list/  
│   ├── facility-detail/  
│   ├── membership-card/  
│   ├── user-registration/  
│   └── top/  
├── components/    # 共通UI  
├── hooks/          # 共通フック  
└── routes/         # ルーティング
```

機能単位でディレクトリを分割

→ AIが「この機能はここ」と判断しやすい

server/ - DDD + クリーンアーキテクチャ

```
src/  
├── domain/  
│   └── model/      # エンティティ  
├── use-case/       # ビジネスロジック  
├── infrastructure/  
│   └── repository/  
├── handler/        # Lambda Entry  
├── presenter/      # レスpons整形  
└── di-container/   # DI設定
```

依存の方向を明確化

→ ESLintで依存関係を強制

→ 変更に強く、移植性も高い

Figma MCP + tailwind.config.js

- Figmaのデザイントークンを `tailwind.config.js` にマッピング
- カラー、フォントサイズ、スペーシングを変数化
- AIがFigmaを参照しながらコンポーネント実装

結果

Figmaと全く同じデザインでフロントエンド構築が可能

400 x 300

```
export default {
  theme: {
    extend: {
      colors: {
        primary: {
          DEFAULT: "#0071BE",
          touch: "#005A98",
        },
        error: {
          DEFAULT: "#C62828",
          touch: "#A12020",
        },
        dark: "#080808",
        default: "#414143",
        soft: "#757578",
        // ... Figmaのカラーパレットと1:1対応
      },
      fontSize: {
        "34ptW3": ["34px", { lineHeight: "41px", fontWeight: "300" }],
        "32ptW6": ["32px", { lineHeight: "38px", fontWeight: "600" }],
        // ... Figmaのタイポグラフィと1:1対応
      },
    },
  },
};
```

Issue/PR/コミットがまたコンテキストになる

Backlog / MTG / 資料



GitHub Issue (詳細な仕様)



コード実装



構造化されたコミット



品質の高いPR・更新されたIssue

まとめ

1. 仕様駆動を行うためには、機能だけでは不十分

KiroやSpec-kitのデフォルトの機能だけでは不十分。プロジェクト全体のコンテキスト（Backlog、MTG、予算、スケジュール等）を含める必要がある

2. 全てをローカルに集約する

APIやMCPに頼らず、ファイルとしてコンテキストを持つことで高速化

3. GitHub Issue/PR/コードをドキュメントとして機能させる

別途仕様書を作るのではなく、開発成果物自体がドキュメントに

4. 既存アセット + rulesyncでさらに高速化

テンプレートリポジトリとカスタムコマンドで品質と速度を両立

開発面

- 手戻りの削減
- 開発速度の大幅向上
- AIとの協働が真の仕様駆動へ

運用保守面

- ドキュメントの品質が高い
- 機能仕様の把握が高速
- メンバー追加時のオンボーディング効率化

チーム面

- 情報の一元化
- ナレッジの蓄積
- 属人化の防止

現在

カスタムコマンドやルールを使って**AIと協働**しながらタスクを実行

- 議事録生成、Issue作成、PR作成...
- 人間がトリガーを引き、AIが実行



目指す姿

Skill / Subagent に移行し、より任せる範囲を広げる

- AIが自律的にタスクを判断・実行
- 人間は最終確認とレビューに集中
- より高度な協働関係へ

classmethod

ご清聴ありがとうございました

全てのコンテキストを集約し、真の仕様駆動を実現しましょう！